

Core Concept 2: Cursor-Based Pagination vs Offset Pagination

If you have ever tried to loop through an API using `page=1`, `page=2`, `page=3` and suddenly the server returns `[]` (empty data) or a 400 Error, you have encountered Cursor-Based Pagination.

The Flaw of Offset Pagination (page=3)

Old APIs used "Offset" pagination. `GET /users?limit=50&offset=100` tells the server: "Skip the first 100 users, give me the next 50." **The problem:** If someone deletes a user while you are reading page 3, all the items shift by 1. Some items get skipped; some items get duplicated. It is also extremely slow for databases to skip million-row offsets.

The Modern Solution: "The Bookmark" (Cursor)

Modern frontend frameworks (React/GraphQL), platforms (Twitter, Instagram), and enterprise APIs (Shopify, Justdial) use **Cursors**.

A Cursor is like a physical bookmark. Instead of telling the server "Give me page 3", the server gives you the data *and* a cryptographically hashed string representing exactly where you left off.

How it looks in the Network Tab

Request 1 (The initial load)

```
// POST /api/search
{
  "category": "Tech shops",
  "cursor": null // We are starting fresh
}
```

Response 1

```
{
  "data": [ { "name": "Shop A" }, { "name": "Shop B" } ],
  "pageInfo": {
    "hasNextPage": true,
    "endCursor": "cXd1cnR5dWlvcGFzZGZnaGprbHo=" // THIS IS THE
    GOLDEN TICKET
  }
}
```

```
}
}
```

If you try to send `{"cursor": null}` again, you will just get Shop A and Shop B forever. You *must* extract that `endCursor` string and embed it into your next request.

Step-by-Step Implementation in Python

To scrape a cursor-based API, you replace a basic `for` loop with a `while` loop that constantly updates its payload based on the *previous* server response.

```
import aiohttp
import asyncio

async def fetch_all_records(start_url):
    all_data = []

    # 1. Initialize the cursor as None (or empty string depending
    on the API)
    current_cursor = None
    has_more_data = True

    async with aiohttp.ClientSession() as session:

        # 2. Keep looping as long as the server says there is more
        data
        while has_more_data:

            # 3. Build the payload. Inject the cursor we saved
            from the LAST loop.
            payload = {
                "query": "Tech shops",
                "limit": 50,
                "cursor": current_cursor
            }

            print(f"🔍 Requesting with cursor:
            {current_cursor[:10] if current_cursor else 'Initial'}")

            async with session.post(start_url, json=payload) as
            response:

                if response.status != 200:
                    print("❌ API rejected the request.")
                    break

                data = await response.json()
```

```

# 4. Save the actual business data
if "data" in data:
    all_data.extend(data["data"])

# 5. Extract the NEW cursor for the next loop!
# This logic changes depending on the specific API
structure
page_info = data.get("pageInfo", {})

if page_info.get("hasNextPage"):
    current_cursor = page_info.get("endCursor")
else:
    # The server said there is no more data. Stop
the loop.

    has_more_data = False
    print("☑ Reached the end of the database!")

    await asyncio.sleep(1) # Be a polite bot

return all_data

if __name__ == "__main__":
    url = "https://example.com/api/graphql"
    asyncio.run(fetch_all_records(url))

```

Key Takeaways for High-End Scraping

1. **You cannot guess Cursors.** They are usually base64 encoded strings of database primary keys or timestamps. You have to wait for the server to give them to you.
2. **Sequential Dependency:** Because you need the result of Page 1 to build the request for Page 2, you cannot use extreme parallel concurrency (like fetching page 1-100 at the exact same time). This forces you to scrape sequentially per-category.
3. **Dead Ends:** Sometimes Anti-Bot systems will intentionally return an invalid or broken cursor to trap scrapers. Always wrap your cursor extraction in `try/except` blocks.